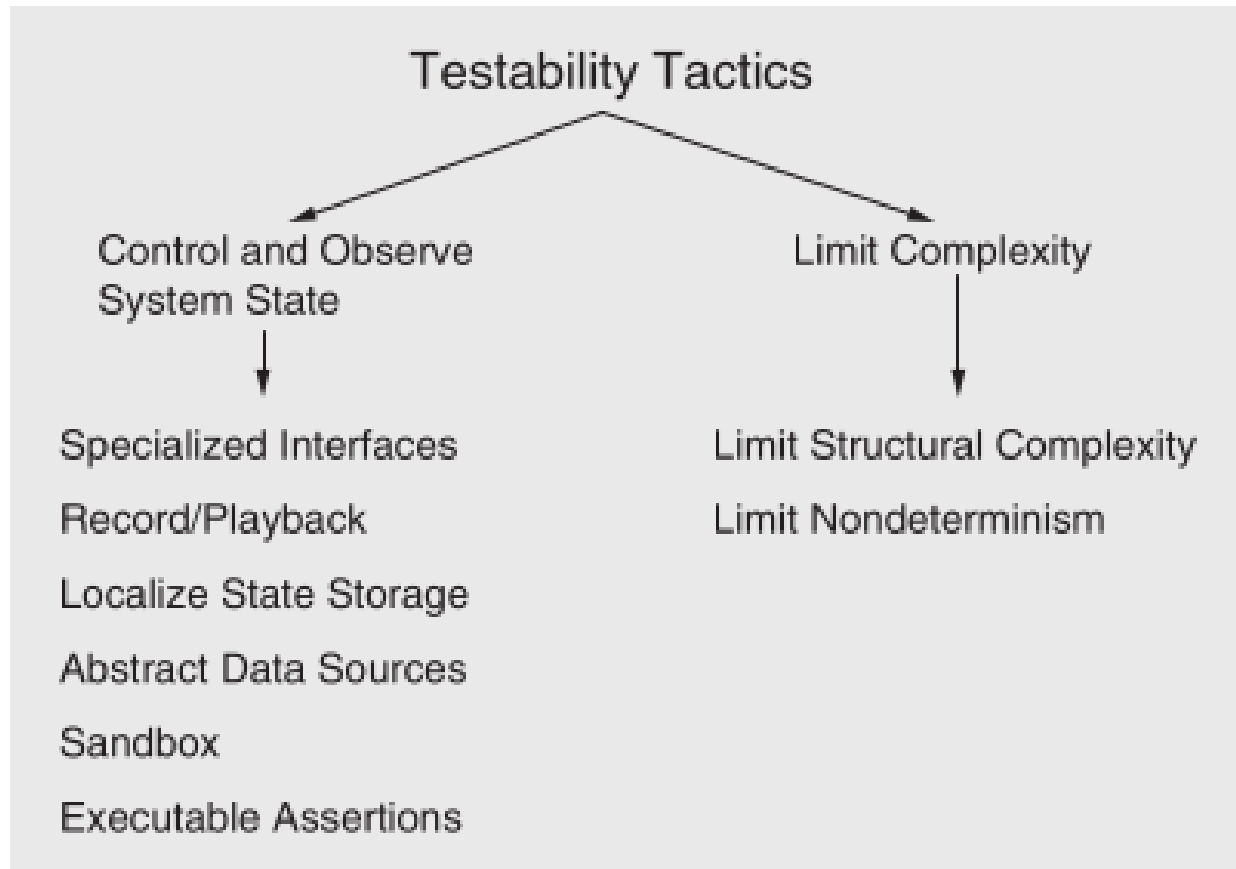


Testability (Comprobabilidad)

Testability

- Facilidad de un sistema para demostrar sus fallos
 - Probabilidad que un sistema falle en la siguiente prueba, dado que tiene una falla
- Requisitos
 - Posibilidad de controlar entradas
 - Posibilidad de observar salidas
 - Posibilidad de observar su estado interno (antes, durante y después de cada prueba)

Tácticas de Comprobabilidad



Controlar y observar el estado del sistema

- Interfaces especializadas
 - Getters, setters, reportar estado completo de un objeto
 - Resetear el estado del objeto
 - Logging
- Record/playback. Guardar y reproducir cambios de estado en el sistema
 - [LiveRecorder](#)
 - [RevDebug](#)
 - [Java Flight Recorder \(JFR\)](#)
 - [RR \(C++\)](#)
 - [OpenReplay](#)

Controlar y observar el estado del sistema

- Localizar (factorizar) el almacenamiento del estado del sistema
 - Implementar máquinas de estados en el código
- Sandboxing y abstracción de fuentes de datos
 - Aislar el componente del mundo real
 - Virtualización
 - Mocks, fuentes de datos simulados, etc.
- Uso de `assert` en ejecución
 - Pre, post condiciones e invariantes

Limitar complejidad

- Reducir complejidad estructural
 - Reducir acoplamiento, reducir profundidad de herencia, reducir polimorfismo
 - Aumentar cohesión
- Reducir no-determinismo
 - Controlar paralelismo (ej. Condiciones de carrera)

Patrones de Comprobabilidad

Inyección de dependencias

- Asegurar que todas las dependencias de puedan parametrizar desde el constructor de una clase
- Ventajas
 - Flexibilidad para cambiar comportamiento del sistema (Mocks)
- Desventajas
 - Complejidad adicional
 - Curva de aprendizaje

“Classic” Programming (no DI)

```
interface Persistence {  
    void save(Object obj);  
}  
  
class XMLPersistence implements Persistence {  
    @Override  
    public void save(Object obj) {  
        // Saves obj in XML format  
    }  
}  
  
class SQLPersistence implements Persistence {  
    @Override  
    public void save(Object obj) {  
        // Saves obj in a SQL database  
    }  
}
```

```
class DataProcessor {  
    XMLPersistence p = new XMLPersistence();  
  
    public void execute() {  
        Data obj = new Data();  
        // manipulate obj's data  
        p.save(obj);  
    }  
}
```

```
DataProcessor proc = new DataProcessor();  
proc.execute();
```

Disadvantages?

Dependency Injection

```
interface Persistence {  
    void save(Object obj);  
}  
  
class XMLPersistence implements Persistence {  
    @Override  
    public void save(Object obj) {  
        // Saves obj in XML format  
    }  
}  
  
class SQLPersistence implements Persistence {  
    @Override  
    public void save(Object obj) {  
        // Saves obj in a SQL database  
    }  
}
```

```
class DataProcessor {  
    Persistence p;  
  
    public DataProcessor(Persistence p) {  
        this.p = p;  
    }  
  
    void execute() {  
        Data obj = new Data();  
        // manipulate obj's data  
        p.save(obj);  
    }  
}
```

```
DataProcessor proc = new DataProcessor(new XMLPersistence());  
proc.execute();
```

Advantages?

Disadvantages?

Patrón estrategia

- Interfaz abstracta
- Implementaciones concretas para diferentes algoritmos
- Ventajas
 - Unificación de interfaces
- Desventajas
 - Ligero aumento de complejidad
 - A veces hace más difícil depuración

Filtros de intercepción

- Código que intercepta una petición
- Extraer información útil para testing
- Ventajas
 - Aumenta cohesión
 - Reduce acoplamiento
 - Factoriza código de logging y auditoría
- Desventajas
 - Overhead